Smart Home Sensor

Project Proposal

Team EcoNest

Sutton Elliott

Shiv Ranchhod

William Xu

Evan Johnston

Tyler Rottermond

Sriharsha Jandhyala

Department of Computer Science

Texas A&M University

2/13/2026

Table of Contents

1	Executive summary	3
2	Introduction	4
2.1	Needs statement	4
2.2	Goal and objectives	4
2.3	Design constraints and feasibility	5
3	Literature and technical survey	6
4	Proposed work	9
4.1	Evaluation of alternative solutions	9
4.2	Design specifications	11
4.3	Approach for design validation	15
5	Engineering standards	17
5.1	Project management	17
5.2	Schedule of tasks, Pert and Gantt charts	19
5.3	Economic analysis	21
5.4	Societal, safety and environmental analysis	21
5.5	Itemized budget	23
6	References	23
7	Appendices	25
7.1	Product datasheets	25
7.2	Bios and CVs	30

1 Executive summary

Smart home technologies have become increasingly common, yet most existing systems rely on **isolated devices** and **static, rule-based** automation that fails to adapt to occupant behavior or provide **meaningful insight**. Despite widespread adoption of smart security products, residential burglaries continue to occur at **high rates**, and homes **without** effective monitoring remain significantly **more vulnerable**. Similarly, while smart energy devices promise **efficiency gains**, inactive and poorly coordinated smart home equipment wastes **billions of dollars** in electricity annually. These shortcomings reveal a critical gap: current smart homes collect data, but **do not reason over it intelligently**.

The goal of this project is to **design and prototype** an **AI-driven** centralized smart home system that uses **machine learning** to analyze security and energy data holistically, enabling **intelligent automation**, anomaly detection, and **personalized decision support**. Rather than relying on predefined thresholds or simple if–then rules, the proposed system **learns** normal household behavior over time and uses that **knowledge** to optimize energy consumption and enhance home security with **minimal user intervention**.

The system architecture is built around a **Raspberry Pi** based edge hub that **aggregates** data from **multiple sensors**, including **smart energy plugs**, **motion sensors**, and **sound sensors** distributed throughout a residence. These sensors **continuously** generate multi-modal data streams describing appliance usage, occupancy patterns, and environmental activity. Raw data is **preprocessed locally** and stored in a **structured SQL database**, ensuring low-latency access and improved privacy while supporting **scalable machine learning workflows**.

Artificial intelligence is the core differentiator of the system. Machine learning models analyze **historical** and **real-time** sensor data to **identify trends, detect abnormal behavior**, and **generate actionable insights**. Time-series forecasting models learn typical energy and activity patterns for each household, enabling **predictions of future usage** and automated adjustments to reduce waste. **Anomaly detection** algorithms flag unusual events such as unexpected appliance power draw, abnormal sound levels, or motion during unoccupied periods, allowing the system to **proactively alert occupants** or **trigger automated responses**. These **AI-driven insights** are continuously **refined** as new data is collected, allowing the system to **adapt** to changing household routines.

Users interact with the system through a **web-based dashboard** that presents **AI-generated insights** rather than raw sensor readings. The interface **highlights** detected **anomalies, predicted trends, and personalized recommendations**, while **real-time push notifications** ensure occupants are immediately informed of potential security risks or inefficiencies. Automation capabilities allow the system to **act on AI decisions**, such as adjusting device states or **notifying users** when human intervention is required.

The expected result of this project is a fully integrated prototype that demonstrates the feasibility and value of **machine learning based reasoning in smart home environments** under

realistic cost and deployment constraints. By transforming **raw sensor data** into contextual **understanding and intelligent action**, the system advances smart home technology from **reactive monitoring to adaptive**, **AI-driven decision making**, offering **meaningful improvements** in security, energy efficiency, and user awareness.

2 Introduction

2.1 Needs statement

Over **94 million** US **households use smart security tools** (Safehome, 2025), yet according to the **FBI** (Federal Bureau of Investigation), **break-ins** still occur **every 26 seconds** (FBI.gov, 2019). Additionally, the latest FBI Uniform Crime Reporting Statistics says that **homes without security** systems are **3 times** more **likely** to be **burglarized**, (Alarms.org, 2019). From an **energy perspective**, smart **thermostats** claim to **reduce costs** from **10-15% per year** (Globalnewswire.com, 2021). However, **inactive** smart home **devices waste** about 1**9 billion dollars annually** (nrdc.org, 2015). Therefore, a **system** that **consolidates** smart devices is **needed** to help **homeowners** better understand and respond to **security risks** and **inefficient energy** use **across** their **smart devices**.

2.2 Goal and objectives

Under normal conditions, it is up to **homeowners** to operate their home in a way that **conserves energy** and **provides security** to its occupants. The problem is that even if they want to conserve energy or be secure, people might not know how, or forget, or maybe are just too lazy to operate their home in a way that does conserve energy and ensures security. Through **reasoning**, homeowners can have their home **automatically operate** in a way that **conserves energy** and **enhances security**.

Thus, the **main goal** of this project is to **use reasoning** on a **centralized smart home system** to **optimize energy** consumption and enhance **security**.

In order to use **reasoning**, we have the following objectives:

- The smart home system should be able to **collect** different types of **data** for the **machine learning** models to **train** and make **predictions** with.
    - By having many **different types** of **data**, the **models** will be better trained to accomplish our **goal**.
- The smart home system should **quickly** and **securely** transfer collected **data**.
    - As we are working with **continuous** and **sensitive data**, we need to be able to **effectively** transfer that to the models.

Once accomplished our **objectives** pertaining to **reasoning** are:

- The system should be able to **detect anomalies** in the collected data.
    - This can be used to **enhance** the **security** of the occupants. They can **detect** power **issues** in appliances and possible **break-ins** and respond properly to ensure our goal is met.
- The system should be able to **analyze trends** in the collected data.
    - This will be used to make **predictions** on how the occupants will operate the home, and **automatically respond** based on these trends.
- The system should be able to provide **recommendations** to the occupants.
    - Using various data sources, the system should be able to provide recommendations to the occupants pertaining to **energy conservation** and **security** of their home.

Other objectives:

- The system should allow occupants to **access** collected **data** and be **informed** of any changes.
    - While we want the home to **automatically** run considering **energy consumption** and **security**, the occupants should know about these changes in case of **wrong predictions** allowing the **occupants** to **change** the responses.

2.3 Design constraints and feasibility

Technical Constraints:

The **security** portion of our **system** relies on **motion** and **sound sensors**, while the **energy-monitoring devices** and **smart plugs** will provide the **data** for each of these focuses on our design. This **data** will be provided to do a **machine learning** based **analysis** of the **security** and **energy** consumption. These **sensors** must reliably **capture** activity **patterns** and **events** that take place within their respective **environments** that can be processed and **stored** in a structured **database**. **Machine learning** techniques are then required to **analyze** this **data**, identify **abnormalities**, and **generate** meaningful **alerts** and **recommendations**. This all will be managed and **viewable** through a **web** or **mobile application**. The technical **design** must **ensure** that the sensor **data** is **accurate** and **consistent** to support the **AI system**, so that the **reasoning** behind it will be **accurate** and **reliable**.

Physical Constraints:

Motion and **sound** sensors will need to be physically **deployable** within **residential areas** and positioned to effectively **capture** room **activity** and not be intrusive after installation. Proper sensor **placement** is **essential** to ensure that the **data** collected **reflects** the actual data based upon the **energy consumption** and **security-related** events. **Poor placement** of these sensors will **negatively** impact the **machine learning** model and in turn the learning **accuracy**. Energy-monitoring devices also need to be placed at appropriate **power outlets** to provide **reliable** usage and energy consumption **data**. All **hardware** must be **compatible** with the overall **system** and non-invasive of the homeowners space. These physical constraints directly affect the data quality and reliability of the analysis of the machine learning model.

Economic Constraints:

The total project **budget** is configured to be $**300**, or approximately $**50** per **person** (6 group members), with a target cost for **each unit** being less than **150** dollars. This **constrains** the selection of the different **components**, namely **sensors** and computing **hardware**, favoring **low-cost** components. In addition to this, we would need to favor lower-cost, commercially available motion, sound, and **energy-monitoring** devices. As a result, the **AI/ML** approach needs to be **lightweight models** and efficient data processing techniques rather than computationally expensive methods. While cost is a constraint, the primary objective remains the idea of demonstrating the feasibility of **AI-driven security** and **energy analysis** within the budget requirements that are put before us.

Temporal Constraints:

The project **timelines** is **10** to **11 weeks**, which requires us to integrate the motion, sound, and energy sensors to allow sufficient data collection to properly **train our machine learning model**. **Hardware** building and **deployment** must be completed within the **first phase** of the project which is about the **first 4 weeks** of the project. This will be around the time of the **first Critical Design Review** to enable us to gather data, **train the model**, and **validate** the **design**. The feasibility of the project depends on **alignment** of the **sensors** and **dataset**, in addition to the **model complexity** within the **available** development **time**.

3 Literature and technical survey

Ring Alarm Pro Security Kit:

The **Ring Alarm** Pro Security Kit is a **smart home security system** designed to **detect intruders** through different **entry ways**, including **doors** and **windows**. The system has a combination of door/window contact sensors, motion detectors, and a centralized base station. The **system integrates** closely with a **Ring** doorbell **system** and provides **real-time updates** and **alerts** to **users** through their mobile devices. Installation is designed to be simple and accessible for technologically illiterate users, which makes it widely used for home security systems.

Pros:

- Well-integrated **security system** that includes **cameras**, **alarms**, and **mobile notifications**
- Easily installable and clear setup that requires little technical expertise
- **Reliable** motion **sensing** for intrusion **detection**
- **Scalability** within the **system** with support for **additional sensors** and **devices**

Cons:

- Focused almost **exclusively** on **security**

- **No energy** monitoring, tracking, or **analysis**
- **Limited** cross-device **intelligence** beyond rule-based alerts
- **No machine learning** used within the system for behavioral or pattern recognition

NTI E-1W Environment Monitoring System

The NTI E-1W Environment Monitoring System is an **environmental monitoring system** that is intended for **larger facilities** and rooms such as **server rooms** or **industrial spaces**. This system **supports** a **wide range** of **sensors** that track and analyze temperature, humidity, liquid detection, smoke, and power status, and **trigger alerts** when **predefined thresholds** are not met. **Data** from the **system** is then **moved over** a **network interface** and can be **accessed remotely** though **web-based dashboards** or integrated into other **monitoring systems**.

Pros:

- **Supports** a **wide variety** of **sensors** throughout the environment
- **Threshold-based monitoring** with alerts for hazardous conditions
- Designed for long-term use
- **Connect** via network with **remote access** and alerts available

Cons:

- Relies on **threshold based rules** that are static and predefined
- **No** threat or intruder **detection**
- **No reliable** way to **integrate** other consumer smart devices
- **Lacks predictive analysis** or **insights**

Ecobee SmartSensor:

The Ecobee SmartSensor is a **wireless sensor** designed to extend the range and functionality of Ecobee smart thermostats by **detecting temperature** and the **amount** of **people** in a room. These sensors enable **HVAC systems** to **adjust** to **heating** and **cooling based** on the amount of **people** within a room instead of a single thermostat. The SmartSensor **communicates** with the **thermostat** and is **battery powered**, allowing flexible placement within the home.

Pros:

- **Room occupancy** and temperature sensing
- **Improves** the HVAC **efficiency** and comfort within the home
- Simple installation and communication
- Displays use of occupancy-based automation

Cons:

- **Limited** to **temperature** and motion **sensing**
- Requires to already be using Ecobee thermostat

- **No security** or intruder **detection**
- **No centralized analytics** beyond energy efficiency

Philips Hue Smart Lighting System:

The Philips Hue Smart Lighting System is a consumer smart lighting platform that provides lightbulbs, motion sensors, and **centralized control** through a **mobile application**. The system **enables users** to **automate** lights based on a **schedule**, occupancy detection and **predefined rules**. The product **communicates wirelessly** through a **centralized hub** and works well with other devices such as a Google Home or Alexa.

Pros:

- **Automation** based on occupancy and **scheduling** for specific devices
- **Reliable** consumer smart home system
- Displays **energy-saving potential** through lighting control
- **Compatibility** with common third-party **devices**

Cons:

- **Focused** on **lighting** devices
- **Mainly** used for **sensing** motion and ambient **light** within the environment
- **Automation** is **rule-based** and not based on trends or **reasoning**

Aqara Smart Home Hub with Multi-Sensor Suite:

The Aqara Smart Home Hub with Multi-Sensor Suite is a **consumer IoT platform** that is designed to **integrate wireless** sensors including but **not limited** to motion **detectors**, door and window **sensors**, temperature and humidity sensors, and presence sensors. These sensors **communicate** with a **centralized hub** that processes and **analyzes data**. Based on this data, it allows users to **create automations** and **receive alerts** through a mobile application.

Pros:

- **Supports** a **wide range** of **sensors** (environmental, security, etc.)
- Low-cost and scalability
- Displays multi-sensor usability within a single ecosystem

Cons:

- **Limited analytics** beyond simple automations and alerts
- Heavily system dependent
- **No machine learning** for trend analysis and predictive insights
- Data integration across energy, security, and environment is shallow

Proposed Design:

The **proposed** Smart Monitoring **System differs** from the **existing** smart home products that are on the market by **prioritizing artificial intelligence** and **machine learning** as the **core** selling **point** of this **system**. This is different from the isolated devices or monitoring that only takes in data for one specific niche or category. Sensor data related to **security events** and **energy usages** will be **collected** and **presented** on a **web** or mobile application, where all the data will be viewable for the user. In addition, the system will also use **machine learning techniques** to analyze the data to identify patterns, detect abnormalities, and **generate** meaningful alerts and recommendations. Unlike the current systems on the market that are primarily rule-based or threshold-based, instead the proposed design will **use** the data to drive **reasoning** to provide **insight** across both home and security and energy efficiency. By focusing on intelligent analysis of past and present data, the system enables **users** to make **informed decisions** and **understand** based on the **full context** instead of simply raw sensor data.

4 Proposed work

4.1 Evaluation of alternative solutions

Preexisting Smart Home Device (Google Nest)

Pros:

- Professional user interface
- Reliable hardware
- **Minimal design** time (more time for ML)
- App supports

Cons:

- **Closed source** system (!)
- Data stored by Google instead of locally
- **Subscription** based system

Microcontroller Based System (ESP32)

Pros:

- Low cost and power consumption
- **Native** wireless **communication** support (WiFi/ Bluetooth)

Cons:

- **Limited** processing **power** (video streaming/ image processing)
- Limited storage
- Difficult to scale

Cloud Based Solution (devices send all information to cloud)

Pros:

- Scalability
- Easy **ML implementation**
- Access from anywhere extremely easily over internet

Cons:

- High dependence on internet
- **Increased latency** for quick actions
- Cloud hosting costs (subscription fees)
- Privacy and security concerns

Separate Energy and Security System

Pros:

- Simpler design for each
- Can optimize independently
- Reduced cost per system

Cons:

- Duplicate hardware might drive costs above savings
- No shared data
- **No** integrated **insights** (relating camera to power usage)

Dedicated NVR with Separate Energy Monitor

Pros:

- NVR is built for security, so minimal design effort spent there
- Reliable and efficient video streaming
- **Reduces** load on **energy** monitoring system

Cons:

- Significantly higher costs
- Limited integration between two systems
- Less flexibility (NVR is not open source)
- **No** integrated **insights** (relating camera to power usage)

Raspberry Pi Smart Home (Proposed Solution)

Pros:

- Computing power sufficient for **data streaming** and **analysis**
- Linux based platform allows for **open sourced** software
- Easy integration of several sensors
- Storing data locally eases privacy and latency concerns

Cons:

- High relative power consumption
- **Complex** system **design**
- **Security risk** if done poorly

Overall, we are choosing to move forward with the option of the **Raspberry Pi** due to its extreme **flexibility** for what we consider to be a very fair price. We can implement some **machine learning trends** in it without breaking the bank, while also having the **energy system**

incorporated **into** the **Pi** allowing for one system. Some **alternatives** include having **two systems** (see NVR), but this would mean having double the setup for an end user, which is **not desirable**. So while the **other systems** can offer **better security** for a **higher cost**, we are **choosing** to run off a **Pi** for its **low cost and scalability**.

4.2 Design Specifications

This section presents the detailed design of the Smart Room Monitor, a **centralized smart home** monitoring **system** focused on **applying reasoning** for **security** and **energy management**. The design is organized into **6 modules** that collectively enable sensor **data collection**, cloud **storage**, **analysis**, and an user **dashboard**. **Figure 1** below provides a **high-level block diagram** of the **full system architecture**, and the subsections after this section describing each module in detail.

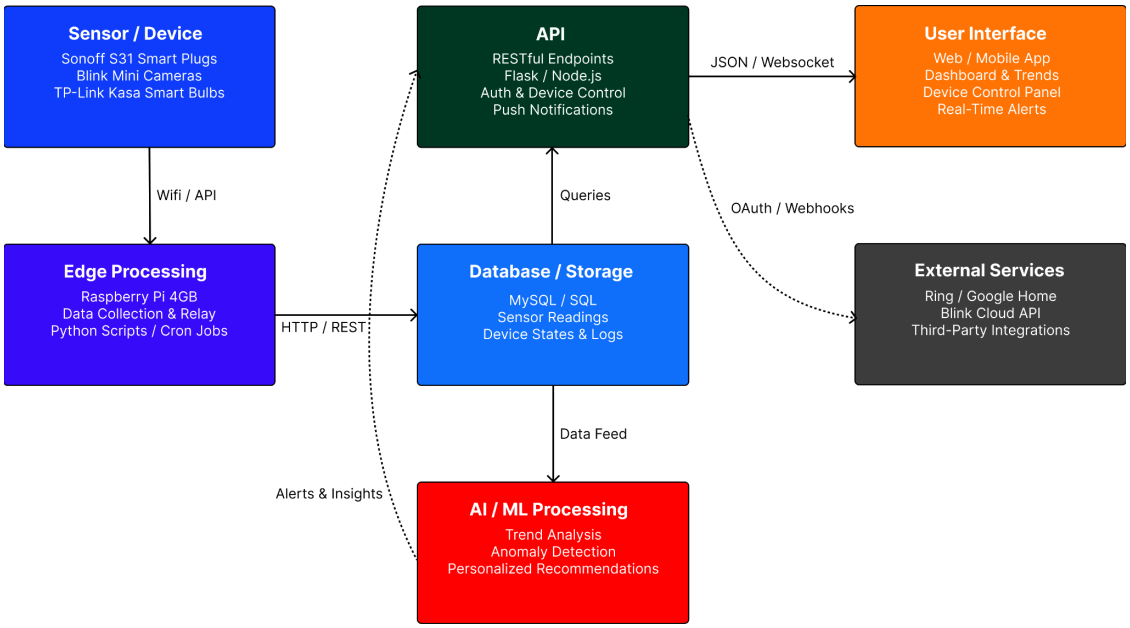

Figure 1: *High-level system architecture of the Smart Room Monitor. Six modules communicate via Wi-Fi and REST APIs to collect sensor data, process it with ML, and present insights to the user.*

4.2.1 Sensor / Device Layer

The **Sensor/Device** Layer is the physical hardware deployed throughout the home. Each per-house unit consists of a **Raspberry Pi 4 Model B** acting as the **central** hub, connected to two Sonoff S31 smart plugs, a sound sensor, and a motion sensor. The system will be **deployed** across **3 houses** for **testing** and **validation**. Figure 2 below illustrates the **hub and spoke architecture** of a single unit.

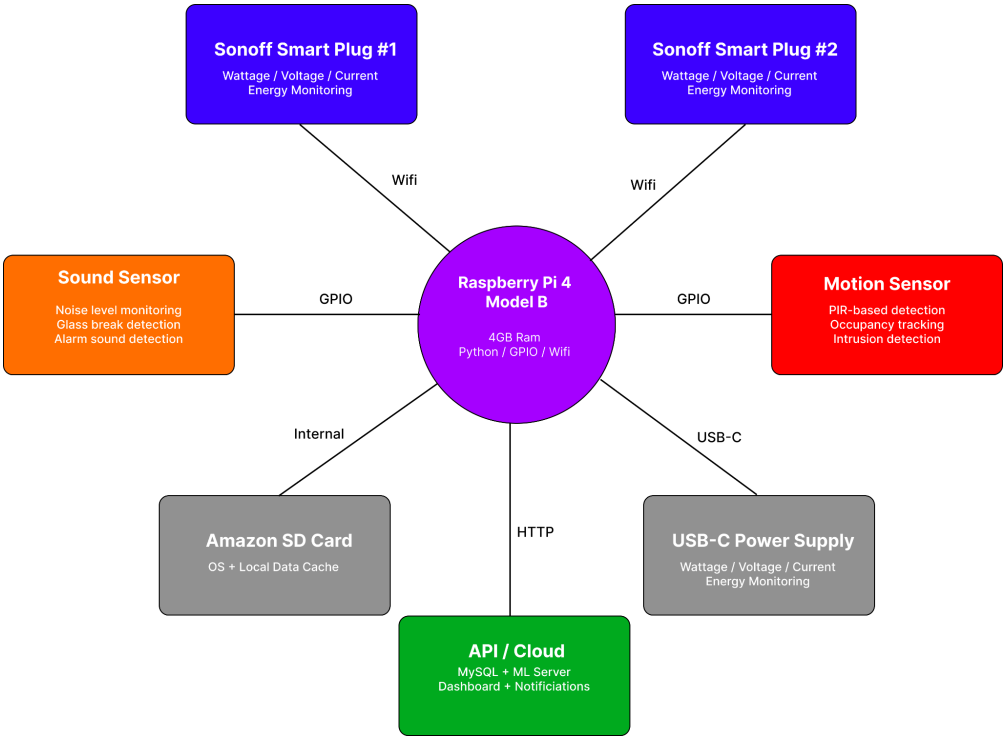

***Figure 2**: Hub and spoke architecture of a single per-house sensor unit. The Raspberry Pi 4 serves as the central hub, communicating with Sonoff S31 smart plugs over Wi-Fi and reading motion and sound sensors via GPIO pins. Data is forwarded to the API/cloud layer over HTTP.*

The Sonoff S31 **plugs** provide **real-time wattag**e, voltage, and current **readings** for each connected appliance via their **local Wi-Fi API**, covering the **energy monitoring** use case. The **motion sensor** detects **movement** within a room to identify unauthorized entry or occupancy patterns, while the **sound sensor** monitors for **abnormal noise levels** that may indicate a security concern such as **glass breaking** or an **alarm** going off. The motion and sound sensors **connect** directly to the Raspberry Pi **via GPIO pins**, while the smart **plugs communicate wirelessly** over the home Wi-Fi network. This layer was designed to use commercially available, off-the-shelf components to keep **costs within** the **$300 budget** while covering the **two primary** use cases of **security** and **energy management**.

4.2.2 Edge Processing (Raspberry Pi)

A **Raspberry Pi 4 Model B** with **4 GB** of **RAM** is used as the local edge-**processing hub** for each house. Running Raspberry Pi OS, it executes **Python scripts** to continuously read **data** from the connected **sensors**. For the Sonoff smart plugs, the Pi uses the **Sonoff LAN library** to **query** energy data over the local network **without** relying on external **cloud services**. The sound and motion **sensors** are **wired** to the Pi's **GPIO pins** and read directly through **Python** libraries. The Pi **preprocesses** raw **readings**, such as converting raw **wattage** samples into **rolling averages** and **filtering noise** from the sound sensor, then forwards the **cleaned data** to the **API layer** via **HTTP POST** requests. This local preprocessing **reduces** unnecessary

network **traffic** and provides a layer of **resilience** if the internet connection is temporarily lost, as **readings** can be **cached locally** on the Pi and **synced** when **connected**.

4.2.3 Database Layer

All **sensor data** is persisted in a **MySQL** relational **database** hosted on the same local network or on a **cloud instanc**e. The **schema** is organized around **three** core **tables**. The first, **device_readings**, **stores** timestamped **energy metrics** including watts, voltage, amperage, and cumulative kilowatt-hours for each smart **plug**. The second, **sensor_events**, logs timestamped **motion** detections and **sound** level **readings** with fields for event type, severity, and room **location**. The third, **device_states**, tracks the **current on/off status** and last-changed timestamp for every device. **Foreign keys link** readings to a **devices table** that stores **metadata** such as **device** name, **room** assignment, and **type**. Indexing on timestamp and device_id **columns ensures** that trend **queries** and anomaly **detection** can scan recent **data** efficiently. The **database** also stores **user preferences** and **notification** history so the **AI module** can **personalize** its **recommendations** over time.

4.2.4 API Layer

The **API layer** is implemented as a **RESTful web service** using **Flask** with **Python**. It creates **endpoints** for **ingesting sensor data** from the Raspberry Pi, **querying** historical **readings** for the dashboard, **issuing** device **control commands**, and registering push-notification tokens. **Authentication** is handled via **JSON Web Tokens** to **ensure** that only **authorized users** and devices can **interact** with the **system**. For **real-time** updates, the server **maintains** a **WebSocket** connection with the **frontend**, pushing new **readings** and **alerts** the **moment** they **arrive** rather than requiring the client to poll. The **API** is designed to be **extensible** so that **third-party integrations** such as Ring or Google Home **could** be **added** in **future** iterations via OAuth 2.0 flows.

4.2.5 AI / ML Processing Module

The **AI/ML module** is the **core differentiator** of the **Smart Room Monitor** and will be **hosted** on a **dedicated laptop**. It pulls **data** from the MySQL **database** at fixed **intervals**, including **energy** readings from device readings and **motion** and **sound** logs from sensor events. Raw sensor data is first passed through a **feature engineering pipeline** that **generates** meaningful **inputs** for the **models**, including **time-of-day** and **day-of-week encoding**, **rolling** power **averages** over 5-minute and 30-minute windows, motion frequency per hour, and sound level variance. These **features feed** into **four key ML capabilities**.

For **trend analysis**, the **module** uses **time-series forecasting** models from **Hugging Face** to **learn** each household's typical energy and **activity** patterns and **predict future usage**. For **anomaly** detection, an isolation forest model **identifies outliers** in **energy** consumption and **unusual motion** or **sound** events that deviate from the **learned baseline**, such as a forgotten appliance drawing power overnight or **unexpected motion** during vacant hours. For **personalized recommendations**, the system uses the **detected patterns** to **generate**

actionable suggestions tailored to each household, such as shifting high-energy appliance usage to off-peak hours or adjusting device schedules based on **observed routines**. For real-time adjustments, the module **continuously evaluates incoming data** against the **trained models** and can trigger immediate **automated responses** through the **API layer**, such as **sending** an **alert** when a **motion** and **sound anomaly** coincide, suggesting a potential **security** event.

Models are **retrained weekly** using the **latest** accumulated **data** to **adapt** to **changing** household **behavior**. **Hugging Face Transformers** and **scikit-learn serve** as the **primary ML frameworks**, with **model artifacts** versioned and stored locally. Anomalies, predictions, and **recommendations** are written back to the database with timestamps, device IDs, confidence scores, severity levels, and **suggested actions**, which the **API layer** then **serves** to the frontend and **notification system**.

4.2.6 User Interface

The **user interface** is a web app **built** with **React** and **served** through the **API layer**. The **dashboard** home screen **displays** an **overview** with **real-time energy consumption** gauges, **motion** and **sound alert** indicators, and device on/off toggles. A **trends** page renders historical charts, allowing users to **visualize energy usage** over days, weeks, or months and compare rooms side by side. The **alerts** page **aggregates** all **notifications chronologically** and lets users acknowledge or snooze them. Push **notifications** are **delivered** via **Firebase** Cloud **Messaging** to both web and mobile browsers, ensuring **users receive critical alerts** even when the app is closed. The interface follows a card-based layout, presenting data as a carousel of **digestible information** cards rather than dense tables. The design will be **prototyped** in **Figma** before development.

4.2.7 Data Flow

Figure 3 shows the **data flow** for a **single** sensor **reading**, from the moment a smart plug **captures** a **wattage** value to the point where the **user** receives an **alert**. This **pipeline** executes in near **real time**, with typical **latency under five seconds** from measurement to notification.

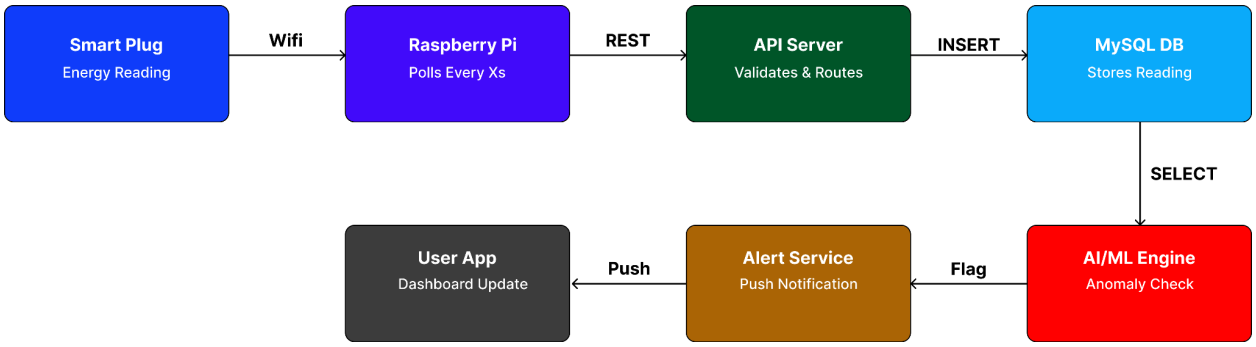

Figure 3: *End-to-end data flow for sensor reading. A smart plug transmits energy data to the Raspberry Pi over Wi-Fi, which forwards it through the REST API to the MySQL database. The ML engine queries the database, flags anomalies, and triggers push notifications or automated device responses.*

4.3 Approach for design validation

This section will **outline** how we will **verify** that our **smart home system** effectively meets the stated **goals** of optimizing **energy consumption** and enhancing **home security** for homeowners. The **validation** process will **evaluate** whether the system successfully **collects** sensor data, **transfers** it securely, **analyzes** it using **reasoning** techniques, and provides useful **recommendations** and **alerts** to homeowners. For thorough evaluation, **hardware** system components will be **individually tested**, followed by **software** validation and **reasoning** functionality tests, and **end-to-end** system **performance** under **practical, realistic** conditions.

4.3.1 Hardware Component Testing

Each **hardware** component of the **system** will be **individually** tested to verify proper **functionality** in the context of **overall** system **performance**.

- **Sonoff Smart Plugs**: Test **accuracy** of voltage, current, and power **readings** under **different** appliance **loads**
- **Sound Sensor**: Simulate **normal** and **abnormal** sound events to **evaluate** detection **reliability**
- **Motion Senso**r: Test home **intruder detection** under **various** movement **conditions**
- **Raspberry Pi 4 Model B**: Verify data **collection/polling**, local **processing**, and effective **communication** with the **API** server
- **USB-C Power Supply**: Evaluate **stability** of power **delivery** to the Raspberry Pi during **continuous** operation
- **Amazon SD Card**: Test data **caching** during Wi-Fi network **outages** and successful **synchronization** afterward

4.3.1 Software and Reasoning Testing

The **software** and **machine learning** portion of the system will be **validated** by testing **data transfer**, model **reasoning** outputs, and generation of **alerts/recommendations** in the context of **energy** optimization and **security**.

- **Data Processing**: Verify **sensor** data is **received**, **formatted** and **transmitted** through the **API** without data **corruption**
- **Database Reliability**: Confirm **data** is stored **accurately** in the **database**, sensor and device **states** are **logged**, and **queries** return information **correctly**
- **AI Anomaly Detection**: Introduce **scenarios** that trigger **alerts/recommendations** (e.g. energy spikes, unexpected motion, loud noise) and **verify** that the ML system **flags** and **responds** to those **events** correctly
- **AI Trend Analysis**: Collect **data** on **hourly** and **multi-day** intervals and validate the system identifies **patterns** in intrusion **detection** and energy **consumption**

- **AI Reasoning**: Confirm the AI **system** generates **actionable** suggestions for reducing **energy** consumption and improving **security** based on trend **analysis/anomalies** and **verify** text output
- **Alerts and Notifications**: Verify any **alerts** or push **notifications** generated are **transmitted** through the **API** and **delivered** to the user **app** in real time
- **User Dashboard and App**: Perform **unit** tests on key user interface **features** in the app and ensure **insights**, sensor/device **data**, and **alerts** are clearly displayed for **interpretation**

4.3.3 End-to-End Testing

End-to-end validation confirms that the **system** operates **correctly** under **realistic** home **conditions**. It **verifies** the **practicality** of the **system** for homeowners and if it is **viable** for achieving the **key** objectives of **improving** the **energy efficiency** and **security** of a given **household**.

- **Full Pipeline Verification**: Trigger multiple events at the sensor level and test simultaneous events (i.e. motion and sound sensor detect abnormalities) and confirm data flows correctly through the Raspberry Pi, API, database, AI/ML processing, and to the user application without any corruption
- **Scenario Testing**: Run multiple **use cases** that reflect the project **objectives** (energy **consumption** and **security**). Each **scenario** will be tested to **verify** a user **alert** or **recommendation** is received within a certain time **interval** that is appropriate for the situation. The following are examples of **use cases**:
    - An appliance plugged into a **smart** plug is left running for multiple hours and drawing too much **power**
    - While a homeowner is away from the house, unexpected **motion** and **sound** levels are **detected** at the front door
    - High sounds levels are **detected** in the house consistent with an appliance **breaking** or an **alarm** sound
- **Latency and Reliability Testing**: Measure **end-to-end** time from **event** occurrence to user **notification** and verify **continuous** monitoring over an **extended** time period
- **Recovery Testing**: Test **Wi-Fi interruption** scenarios for brief and long time **intervals** and confirm the system locally **caches** data, resumes normal data **uploads**, and correctly generates **recommendations** and **alerts** without significant, permanent **data** loss

5 Engineering standards

5.1 Project management

Evan Johnston

Database and Backend Engineer

- Designs and maintains the MySQL database
- Manages data transfer between the Raspberry Pi and machine learning system through API's

Qualifications

- Experience with various databases including PostgreSQL and MySQL as well as non-SQL databases such as Redis and MongoDB.
- Understanding of database and software relationship.

Sriharsha Jandhyala

Machine Learning Engineer

- Designs and implements AI models for anomaly detection and trend analysis
- Using energy and security data collected from smart devices, will apply logical reasoning to recommendations to users

Qualifications

- Experience with machine learning libraries (PyTorch, Transformers) and fine-tuning LLMs (Mistral 7B)
- Experience with PostgreSQL/MySQL and AWS
- API Development

Tyler Rottermond

Hardware and Embedded System Engineer

- Responsible for selecting, configuring, and deploying physical components such as the Raspberry Pi, smart plugs, cameras, and sensors
- Ensures reliable connectivity, power management, and proper operation within the home environment.

Qualifications

- Development on Raspberry Pi and other linux environments
- Sensor data acquisition and interfacing (I^2C, SPI, UART)
- Data preprocessing and filtering for cloud-based systems
- REST API integration and secure data transmission (HTTP/HTTPS, JSON)

Sutton Elliott

Machine Learning and Hardware Integration Engineer

- Designs and implements AI models for tailored recommendations
- Bridge between the sensor data and machine learning model

Qualifications

- Experience with Raspberry Pi and embedded systems
- Background in machine learning
- Knowledge of Azure AI and cloud-based ML services
- Familiarity with SQL databases, and hardware/software integration

William Xu

Frontend Software Engineer

- Develops the User interface that presents sensor data, trends, and recommendations with applied reasoning the the users
- API Design

Experience

- Designed user-centered interfaces
- Experience designing APIs through a centralized system
- Architecture design experience

Shiv Ranchhod

Lead Systems Engineer

- Integration with hardware, software, and machine learning
- Responsible for Risk Management & conveying technical and project deadlines

Qualifications

- Experience with management softwares such as, Monday.com
- Full stack development experience
- IoT and Network experience

When it comes to **keeping track of progress** we will be utilizing **Monday.com** to keep a **schedule** and make sure that we are adhering to the **timeline**. When it comes to **managing** this **software** the **Lead Systems Engineer** will take care of this to make sure to keep the team **updated** on the **deadline** coming up. **Brainstorming sessions** will occur on **Monday** and **Wednesday** during the allotted time for the capstone project. If we need to meet up more this will be decided through **communication** with the team. This will be primarily done on **Slack**.

5.2 Schedule of tasks, Pert and Gantt charts

Any **delays** in the **Hardware setup** will directly **affect** the data **collection**, which will **push everything back** from the AI/ML development to the testing.

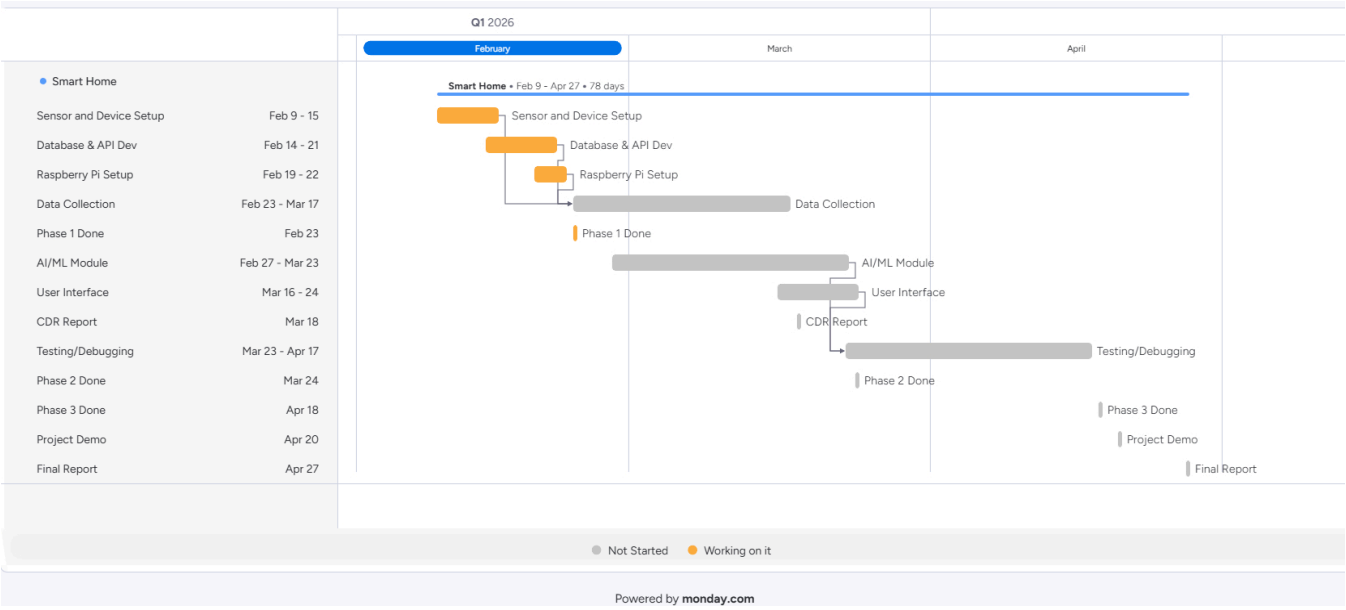

Figure 4: *Gantt Chart with Clear Dependencies*

To summarize, our **first phase** will consist of **setting** up the **hardware** and **database** to get ready for **data collection**. **Phase two** will consist of **setting up** the **machine learning model** and getting it ready to **train off** the **data** that will be **collected** from the environments. During this time, we will also **start** on our **user interface** and **test API's** to make sure that **data** is being **properly displayed**. **Phase 3** will consist of the **testing** and **debugging** while trying to get **everything** to **work** as a **whole**. With the **data** being **analyzed** and **users** being **shown recommendations** with **reasonings applied** to them.

Phase 1 – Core Infrastructure (Feb 9 – Feb 23)

1. **Sensor and Device Setup (Feb 9 – 15)**
    - Procure hardware
    - Install and wire sensors
    - Verify sensor signal output
2. **Database and API Development (Feb 14 – 21)**
    - Design schema
    - Implement backend endpoints
    - Configure cloud connection
3. **Raspberry Pi Setup (Feb 19 – 22)**
    - OS configuration
    - Sensor interfacing
    - API connectivity test

Phase 1 Milestone (Feb 23)

Phase 2 – Data and Intelligence Layer (Feb 23 – Mar 24)

1. **Data Collection (Feb 23 – Mar 17)**
    - Logging sensor streams
    - Cleaning and formatting data
    - Initial dataset validation
2. **AI/ML Module (Feb 27 – Mar 23)**
    - Feature extraction
    - Model selection
    - Training and tuning
    - Local integration testing
3. **User Interface (Mar 16 – 24)**
    - Dashboard layout
    - Alert logic integration
    - Backend connection

CDR Report (Mar 18)

Phase 2 Milestone (Mar 24)

Phase 3 – Integration and Delivery (Mar 23 – Apr 27)

1. **Testing and Debugging (Mar 23 – Apr 17)**
    - System integration testing
    - Performance validation
    - Edge case handling

Phase 3 Milestone (Apr 18)

Project Demo (Apr 20)

Presentation preparation

Demo environment setup

Final Report (Apr 27)

5.3 Economic analysis

The Smart Room Monitoring System demonstrates **strong economic viability** as a **potential** commercial **product** within the **growing smart home market**. While the **prototype** unit **cost** is approximately $**133** due to low-volume component sourcing and the use of a general-purpose Raspberry Pi, expected **production costs** would **decrease significantly** at scale. In a volume-manufactured design, the **Raspberry Pi** could be **replaced with** a custom embedded Linux System-on-Chip (**SoC**), and sensors could be consolidated onto a single printed circuit board, **reducing** the estimated per-unit **cost** to approximately **$60–$80**. This **pricing** positions the system **competitively** with existing consumer smart home hubs while **offering** additional **value** through **integrated AI-driven reasoning** across **security** and **energy** data. By **avoiding** mandatory cloud **subscriptions** and emphasizing local data processing, the system also **reduces** long-term operating **costs** for users, increasing its attractiveness in the consumer market.

From a **sustainability and manufacturability** perspective, the system **relies** on standardized, commercially **available components** sourced from multiple vendors, **minimizing** supply-chain **risk** and ensuring **long-term** part **availability**. **Sensor tolerances** primarily impact **sensitivity** rather than **functional correctness**, allowing calibration and software filtering to compensate for component variation and **improve production yield**. Manufacturing **testability** can be **achieved** through **automated validation** of sensor **outputs**, wireless **connectivity**, and device control **functionality**. **Regulatory compliance** requirements, such as **FCC** certification for wireless communication, are simplified by the use of **pre-certified radio modules**. **Long-term maintenance** is largely **software-based**, including firmware updates and **periodic machine learning model retraining**, which **extends** product **lifespan** and **reduces** electronic **waste**. Overall, the system is well-suited for **scalable manufacturing** and **sustainable** deployment as a consumer smart home product.

5.4 Societal, safety and environmental analysis

The proposed **Smart Room Monitoring System** has several societal implications, both **beneficial** and some things we **need to be aware of** as we are building our design. From a **societal** perspective, the system works to **improve quality of life** of the user by translating **raw motion, sound, and energy sensor data into actionable insights through machine learning** analysis. Rather than requiring users to interpret the readings of each device individually, the system provides **intelligent recommendations with reasoning** from the machine learning model. This will allow for the user to **enhance** their **home safety** and **energy awareness**. By **tracking patterns and anomalies** within the home, the system reduces the burden of constant monitoring while **promoting proactive decision-making** related to both **energy efficiency** and **home security**.

However, the integration of **artificial intelligence** introduces some **ethical** considerations we need to be aware of. **Machine learning models analyze sensitive homeowner and behavioral data** derived from motion and sound sensors, which raises concerns related to the

homeowners privacy. In addition to this, centralized storage of this data, if mishandled, could expose sensitive information about the **households activity patterns**. Users may also over-rely on **AI-generated recommendations** without full understanding of the limitation of the model. To address these concerns, user data access must be **restricted through authenticated** methods. The **AI outputs** must also be designed to provide **reasonable, explainable recommendations** rather than **generic automated decisions**.

There are also some **safety risks** that are associated with the use of smart plugs and wireless sensors in household areas. **Electrical safety** must be considered when integrating energy-monitoring devices into home outlets. **Cybersecurity risks** also exist if the system is **improperly configured or handled**. This could potentially allow for unauthorized access to the device controls or the stored data. Continuing this idea, **incorrect AI/ML predictions** could also result in **false alerts or missed abnormalities**. **Physical safety risks** related to installation are **minimal** due to the use of low-power, wireless, commercially available devices that do not require invasive wiring.

The **environmental** impact of the system is **primarily positive** in nature. The design relies on low-power readily available components to **minimize additional energy expenditure**. Through **machine learning based trend and prediction analysis**, the system **identifies inefficient energy usage** patterns and abnormal energy consumption. This helps to call out and identify behaviors that **may not be obvious from raw readings.** By notifying users of unused or inefficient devices, the system will aim to **reduce any idle energy consumption** and overall household energy waste. This will result in a **net reduction in energy consumption** over time.

To **mitigate ethical, safety, and environmental risks**, several strategies will be incorporated into the design. Sensor data is to be **stored and processed locally** whenever possible to **reduce exposure** to external systems. Secure authorization methods will also be used to restrict access to **authorized users only.** This will minimize cybersecurity vulnerabilities. **AI/ML reasoning** is used as a **decision tool** rather than fully autonomous control, ensuring that users **retain final authority** over actions taken within their home. Finally, the system offsets its own computational energy usage by actively **applying AI-driven analysis** to reduce household energy waste. This will also support **long-term sustainability**.

5.5 Itemized budget

Component	Amount	Amount Per Unit	Cost Per Unit	Subtotal
Raspberry Pi 4 Model B (4 GB)	1	1	$75.00	$75.00
Sonoff S31 Smart Plug (Energy Monitoring)	2	1	$10.00	$20.00
Sound Sensor	1	5	$6.49	$6.49
Motion Sensor	1	5	$9.99	$9.99
Amazon SD Card	1	2	$13.50	$13.50
Raspberry Pi 15W USB-C Power Supply	1	1	$8.00	$8.00
Cost for a unit				$132.98

6 References

Here you acknowledge all the documents that you used as references throughout the text. Please use the IEEE reference format

https://ieeeauthorcenter.ieee.org/wp-content/uploads/IEEE-Reference-Guide.pdf

National Resources Defense Council. *Home Idle Load Devices Wasting Huge Amounts of Electricity When Not Active in Use*. NRDC, https://www.nrdc.org/resources/home-idle-load-devices-wasting-huge-amounts-electricity-when-not-active-use.

“Burglary Statistics.” *Alarms.org*, https://www.alarms.org/burglary-statistics/.

GlobeNewswire. “Home Automation Market Is Poised to Skyrocket to Reach Staggering Valuation of USD 715.6 Billion by 2032: High Demand for Smart Home Energy Management Devices Says Astute Analytica.” *GlobeNewswire*, 9 May 2024, https://www.globenewswire.com/news-release/2024/05/09/2878979/0/en/Home-Automation-Market-is-Poised-to-Skyrocket-to-Reach-Staggering-Valuation-of-USD-715-6-Billion-By-2032-High-Demand-for-Smart-Home-Energy-Management-Devices-Says-Astute-Analytica.html.

Federal Bureau of Investigation. “Crime Clock.” *Crime in the U.S. 2019*, https://ucr.fbi.gov/crime-in-the-u.s/2019/crime-in-the-u.s.-2019/topic-pages/crime-clock.

Delforge, Pierre. “Home Idle Load: Devices Wasting Huge Amounts of Electricity When\ Not in Active Use.” NRDC, 14 July 2015, www.nrdc.org/resources/home-idle-load-devices-wasting-huge-amounts-electricity-when-not-active-use.

Ring. *Ring Alarm Pro Security System*. Ring, Inc., https://ring.com/products/security-system-alarm-pro.

Network Technologies Inc. *E-1W Environment Monitoring System*. Network Technologies Inc., https://www.networktechinc.com/environment-monitoring.html.

ecobee. *SmartSensor for Room Temperature and Occupancy*. ecobee Inc., https://www.ecobee.com/en-us/sensors/smart-temperature-occupancy-sensor/.

Signify. *Philips Hue Smart Lighting System*. Signify Holding, https://www.philips-hue.com/en-us.

Aqara. *Aqara Smart Home Hub and Sensor Ecosystem*. Aqara LLC, https://www.aqara.com/us/products/.

7 Appendices

7.1 Product datasheets

Raspberry Pi 4 Model B

4 Electrical Specification

Caution! Stresses above those listed in Table 2 may cause permanent damage to the device. This is a stress rating only; functional operation of the device under these or any other conditions above those listed in the operational sections of this specification is not implied. Exposure to absolute maximum rating conditions for extended periods may affect device reliability.

Symbol	Parameter	Minimum	Maximum	Unit
VIN	5V Input Voltage	-0.5	6.0	V

Table 2: Absolute Maximum Ratings

Please note that VDD_IO is the GPIO bank voltage which is tied to the on-board 3.3V supply rail.

Symbol	Parameter	Conditions	Minimum	Typical	Maximum	Unit
V_{IL}	Input low voltage[a]	VDD_IO = 3.3V	0	-	0.8	V
V_{IH}	Input high voltage[a]	VDD_IO = 3.3V	2.0	-	VDD_IO	V
I_{IL}	Input leakage current	TA = +85°C	-	-	10	μA
C_{IN}	Input capacitance	-	-	5	-	pF
V_{OL}	Output low voltage[b]	VDD_IO = 3.3V, IOL = -2mA	-	-	0.4	V
V_{OH}	Output high voltage[b]	VDD_IO = 3.3V, IOH = 2mA	VDD_IO - 0.4	-	-	V
I_{OL}	Output low current[c]	VDD_IO = 3.3V, VO = 0.4V	7	-	-	mA
I_{OH}	Output high current[c]	VDD_IO = 3.3V, VO = 2.3V	7	-	-	mA
R_{PU}	Pullup resistor	-	18	47	73	kΩ
R_{PD}	Pulldown resistor	-	18	47	73	kΩ

[a] Hysteresis enabled
[b] Default drive strength (8mA)
[c] Maximum drive strength (16mA)

Table 3: DC Characteristics

Pin Name	Symbol	Parameter	Minimum	Typical	Maximum	Unit
Digital outputs	t_{rise}	10-90% rise time[a]	-	TBD	-	ns
Digital outputs	t_{fall}	90-10% fall time[a]	-	TBD	-	ns

[a] Default drive strength, CL = 5pF, VDD_IO = 3.3V

Table 4: Digital I/O Pin AC Characteristics

Figure 2: Digital IO Characteristics

4.1 Power Requirements

The Pi4B requires a good quality USB-C power supply capable of delivering 5V at 3A. If attached downstream USB devices consume less than 500mA, a 5V, 2.5A supply may be used.

5 Peripherals

5.1 GPIO Interface

The Pi4B makes 28 BCM2711 GPIOs available via a standard Raspberry Pi 40-pin header. This header is backwards compatible with all previous Raspberry Pi boards with a 40-way header.

5.1.1 GPIO Pin Assignments

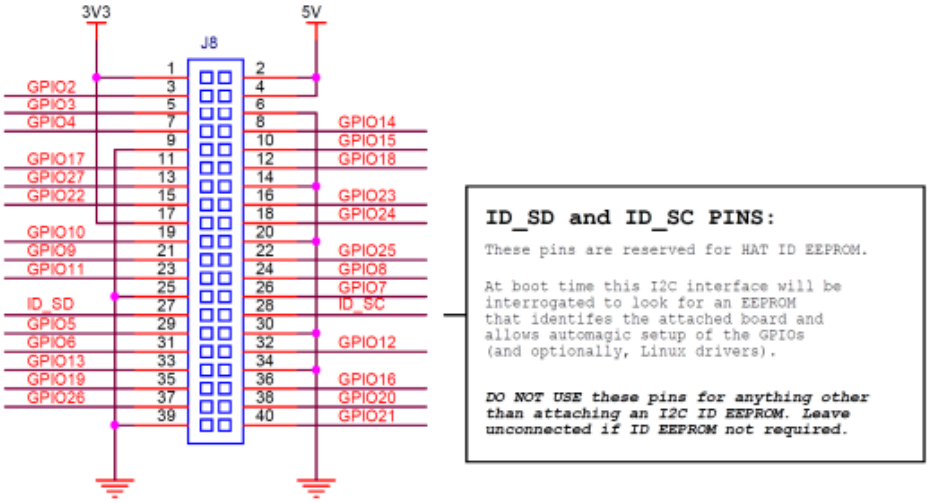

Figure 3: GPIO Connector Pinout

As well as being able to be used as straightforward software controlled input and output (with programmable pulls), GPIO pins can be switched (multiplexed) into various other modes backed by dedicated peripheral blocks such as I2C, UART and SPI.

In addition to the standard peripheral options found on legacy Pis, extra I2C, UART and SPI peripherals have been added to the BCM2711 chip and are available as further mux options on the Pi 4. This gives users much more flexibility when attaching add-on hardware as compared to older models.

5.1.2 GPIO Alternate Functions

GPIO	Default Pull	ALT0	ALT1	ALT2	ALT3	ALT4	ALT5
0	High	SDA0	SA5	PCLK	SPI3_CE0_N	TXD2	SDA6
1	High	SCL0	SA4	DE	SPI3_MISO	RXD2	SCL6
2	High	SDA1	SA3	LCD_VSYNC	SPI3_MOSI	CTS2	SDA3
3	High	SCL1	SA2	LCD_HSYNC	SPI3_SCLK	RTS2	SCL3
4	High	GPCLK0	SA1	DPI_D0	SPI4_CE0_N	TXD3	SDA3
5	High	GPCLK1	SA0	DPI_D1	SPI4_MISO	RXD3	SCL3
6	High	GPCLK2	SOE_N	DPI_D2	SPI4_MOSI	CTS3	SDA4
7	High	SPI0_CE1_N	SWE_N	DPI_D3	SPI4_SCLK	RTS3	SCL4
8	High	SPI0_CE0_N	SD0	DPI_D4	-	TXD4	SDA4
9	Low	SPI0_MISO	SD1	DPI_D5	-	RXD4	SCL4
10	Low	SPI0_MOSI	SD2	DPI_D6	-	CTS4	SDA5
11	Low	SPI0_SCLK	SD3	DPI_D7	-	RTS4	SCL5
12	Low	PWM0	SD4	DPI_D8	SPI5_CE0_N	TXD5	SDA5
13	Low	PWM1	SD5	DPI_D9	SPI5_MISO	RXD5	SCL5
14	Low	TXD0	SD6	DPI_D10	SPI5_MOSI	CTS5	TXD1
15	Low	RXD0	SD7	DPI_D11	SPI5_SCLK	RTS5	RXD1
16	Low	FL0	SD8	DPI_D12	CTS0	SPI1_CE2_N	CTS1
17	Low	FL1	SD9	DPI_D13	RTS0	SPI1_CE1_N	RTS1
18	Low	PCM_CLK	SD10	DPI_D14	SPI6_CE0_N	SPI1_CE0_N	PWM0
19	Low	PCM_FS	SD11	DPI_D15	SPI6_MISO	SPI1_MISO	PWM1
20	Low	PCM_DIN	SD12	DPI_D16	SPI6_MOSI	SPI1_MOSI	GPCLK0
21	Low	PCM_DOUT	SD13	DPI_D17	SPI6_SCLK	SPI1_SCLK	GPCLK1
22	Low	SD0_CLK	SD14	DPI_D18	SD1_CLK	ARM_TRST	SDA6
23	Low	SD0_CMD	SD15	DPI_D19	SD1_CMD	ARM_RTCK	SCL6
24	Low	SD0_DAT0	SD16	DPI_D20	SD1_DAT0	ARM_TDO	SPI3_CE1_N
25	Low	SD0_DAT1	SD17	DPI_D21	SD1_DAT1	ARM_TCK	SPI4_CE1_N
26	Low	SD0_DAT2	TE0	DPI_D22	SD1_DAT2	ARM_TDI	SPI5_CE1_N
27	Low	SD0_DAT3	TE1	DPI_D23	SD1_DAT3	ARM_TMS	SPI6_CE1_N

Table 5: Raspberry Pi 4 GPIO Alternate Functions

Table 5 details the default pin pull state and available alternate GPIO functions. Most of these alternate peripheral functions are described in detail in the BCM2711 Peripherals Specification document which can be downloaded from the hardware documentation section of the website.

5.1.3 Display Parallel Interface (DPI)

A standard parallel RGB (DPI) interface is available the GPIOs. This up-to-24-bit parallel interface can support a secondary display.

5.1.4 SD/SDIO Interface

The Pi4B has a dedicated SD card socket which supports 1.8V, DDR50 mode (at a peak bandwidth of 50 Megabytes / sec). In addition, a legacy SDIO interface is available on the GPIO pins.

5.2 Camera and Display Interfaces

The Pi4B has 1x Raspberry Pi 2-lane MIPI CSI Camera and 1x Raspberry Pi 2-lane MIPI DSI Display connector. These connectors are backwards compatible with legacy Raspberry Pi boards, and support all of the available Raspberry Pi camera and display peripherals.

5.3 USB

The Pi4B has 2x USB2 and 2x USB3 type-A sockets. Downstream USB current is limited to approximately 1.1A in aggregate over the four sockets.

5.4 HDMI

The Pi4B has 2x micro-HDMI ports, both of which support CEC and HDMI 2.0 with resolutions up to 4Kp60.

5.5 Audio and Composite (TV Out)

The Pi4B supports near-CD-quality analogue audio output and composite TV-output via a 4-ring TRS 'A/V' jack.

The analog audio output can drive 32 Ohm headphones directly.

5.6 Temperature Range and Thermals

The recommended ambient operating temperature range is 0 to 50 degrees Celsius.

To reduce thermal output when idling or under light load, the Pi4B reduces the CPU clock speed and voltage. During heavier load the speed and voltage (and hence thermal output) are increased. The internal governor will throttle back both the CPU speed and voltage to make sure the CPU temperature never exceeds 85 degrees C.

The Pi4B will operate perfectly well without any extra cooling and is designed for sprint performance - expecting a light use case on average and ramping up the CPU speed when needed (e.g. when loading a webpage). If a user wishes to load the system continually or operate it at a high temperature at full performance, further cooling may be needed.

7.2 Bios and CVs

Evan Johnston is a senior computer engineering student at Texas A&M University. He first started out his college career as a part of the Texas A&M Engineering Academy, a dual enrollment program partnered with community colleges. He first started attending Texas A&M full-time in the second semester of sophomore year. He quickly displayed his skills getting offered to work as a Peer Teacher for the Department of Computer Science, a program of successful students who provide assistance to other students within the computer science curriculum. Through this job, he was then offered to work as a Teaching Assistant for an introductory C programming course. He has made it his mission to diversify his skillset taking courses from diverse disciplines such as databases, machine learning, computer graphics and animation, etc.

EVAN JOHNSTON

Houston, Texas 77095 • (936) 217-4952 • evanjohnston0426@gmail.com

EDUCATION

Texas A&M University – College Station, Texas

Degree: Bachelor of Science in Computer Engineering **May 2026**

- GPA: 3.959
- Dean's Honor Roll: **Spring & Fall 2024**

Skills

Languages & Frameworks: C/C++, Python, Java, Swift, React, React Native, HTML, CSS, JavaScript, PostgreSQL, SQL, OpenGL, PyTorch, Flask

Systems & Tools: Git, Linux/Ubuntu, AWS, ARMv8, x86-64, Verilog

Additional Skills: front-end, back-end, full-stack, version control, databases, agile development, web development, mobile development, software development life cycle, Rendering, machine learning

Relevant Courses: Machine Learning, Databases, Computer Graphics, Foundations of Software Engineering, Data Structures and Algorithms, Intro to Digital System Design, Program Design and Concepts, Computer Architecture and Design, Electrical Circuit Theory, Computer Systems, Linear Algebra

PROJECTS

Book Store Application – MySQL/Python/Flask **Fall 2025**

- Designed and developed a full-stack software application to support the sale and rental of books.
- Implemented a relational database using MySQL to manage inventory, users, and transactions.
- Built RESTful backend functionality in Flask to handle business logic, authentication, and data persistence.

Bubble Tea Point-of-Sale System – AWS/PostgreSQL/Java **Spring 2025**

- Designed and deployed a **database-driven POS system** for business operations within a 5-person team.
- Implemented backend functionality for **order management, inventory, and menu updates.**
- Deployed on AWS with a PostgreSQL backend to simulate real-world scalability.

Profile Web Page – HTML/CSS/JavaScript **Spring 2025**

- Built and styled a responsive **personal portfolio webpage**.
- Added a **theme-switching feature** for improved user experience and design variety.

WORK EXPERIENCE

Texas A&M University – College Station, Texas **January 2025 - Present**

Teaching Assistant for Department of Computer Science

- Leads weekly labs, reinforcing programming and software engineering concepts.
- Provide academic support to 25+ students through office hours and guided problem-solving.
- Evaluate assignments and deliver feedback to improve student's technical skills

Texas A&M University – College Station, Texas **August 2024 - Present**

Peer Teacher for Department of Computer Science

- Assist students with understanding core computer science concepts and coursework.
- Provide technical guidance and support on course-related software and tools.
- Reinforce and clarify course material through one-on-one and small group instruction, collaborating with faculty to support student success.

ACTIVITIES

Aggie Coding Club – Member College Station, Texas **August 2024 - Present**

- Attend workshops and company talks within various fields of computer science.
- Networking with other students.
- Participating in thought of projects

Shiv K. Ranchhod is a senior Computer Engineering student at Texas A&M University pursuing a minor in Cybersecurity. His academic and professional focus centers on cybersecurity engineering, regulatory compliance frameworks, embedded systems, and hardware-software integration. He most recently served as a Cyber Strategy, Risk, and Compliance Intern at PricewaterhouseCoopers, where he developed structured NIST CSF 2.0 interview frameworks, completed governance scorecards, and contributed to cybersecurity capability modeling initiatives. Previously, he worked as an ETL Developer at Clairvoyance LLC, designing real-time data pipelines using Python and Pandas to enhance data traceability and operational reliability. His technical experience also includes leading the development of a Langflow-based AI regulatory compliance agent and conducting advanced malware analysis involving anti-debugging bypass and network traffic inspection. Shiv is proficient in Python, C/C++, Linux environments, and cybersecurity analysis tools.

Shiv K Ranchhod

361-413-5966 | ranchhodshiv@gmail.com | linkedin.com/in/shivranchhod/ | https://github.com/Shivacadoo

Education

Texas A&M University — College Station, TX
Bachelor of Science in Computer Engineering, Minor in Cybersecurity | GPA: 3.65 — *Aug 2022 – May 2026*

Computer Engineering Experience

Cyber Strategy, Risk, & Compliance Intern — May 2025 – Aug 2025
PricewaterhouseCoopers (PwC) — *Dallas, TX*

- Developed and delivered 15+ interview decks mapped to NIST CSF 2.0 subtopics, standardizing stakeholder interviews and ensuring comprehensive coverage of all control areas
- Completed 5+ NIST CSF scorecards with a focus on Governance functions and enterprise risk management, delivering actionable insights to strengthen organizational risk posture
- Reviewed and analyzed 50+ client-provided documents to complete the document request list and mapped each artifact to relevant NIST CSF 2.0 subcategories, streamlining the scoring process and enhancing framework alignment
- Contributed to the design of a cybersecurity capabilities model and organizational structure for a client by synthesizing insights from internal and client meetings into actionable strategies, supporting decision-making and implementation planning

ETL Developer — Dec 2023 – Jun 2024
Clairvoyance LLC — *College Station, TX*

- Developed a real-time ETL pipeline in pycharm to extract data from websockets, ensuring continuous data availability using heartbeat signals
- Employed Pandas library in Python to manipulate and append timestamps to data frames, facilitating time-series analysis and ensuring data traceability with 75% accuracy
- Engineered and integrated alert systems within the ETL framework to monitor data anomalies and system health, improving data integrity and operational reliability by 8%

Side Projects

AI Agent: RegCom | *Langflow, Python, LLM, Embedment, PwC* — Jun 2025 – Aug 2025

- Spearheaded end-to-end design of a Langflow-based AI agent, orchestrating flow architecture to deliver dynamic cybersecurity compliance guidance based on NIST SP 800-53 Rev 5 and global data-privacy regulations
- Conducted rigorous testing and validation, simulating diverse compliance scenarios and iterating on prompt engineering to improve recommendation accuracy by over 20% while maintaining robust data-privacy controls
- Optimized document ingestion pipeline by configuring text-splitting parameters (chunk size/overlap) and custom parser templates, enabling semantic retrieval from static PDFs
- Led a four-person team to design and implement an AI-driven regulatory compliance agent, presenting the solution to firm partners and directors

Malware Analysis Project | *Ghidra, OllyDbg, x64dbg, Wireshark, Procmon* — May 2025

- Detected and bypassed anti-debugging mechanisms using NOP insertion in OllyDbg and x64dbg to reveal deeper execution flow
- Analyzed encrypted network traffic with Wireshark, exposing Command & Control (C2) attempts and data exfiltration patterns
- Audited registry and file system activity to identify persistence mechanisms and assess overall system security posture

Cybersecurity NFC Scanner | *Python, Network Basics, Soldering, Raspberry PI* — Apr 2024

- Streamlined real-time data synchronization, linking the scanner to a GitHub-based repository for immediate comparison with a catalog of malicious tags having at least 80% success
- Applied advanced troubleshooting techniques to resolve hardware and software integration issues, enhancing system functionality and reliability by 84%

Skills and Organizations

Organizations: PAID, Aggie Coding Club, BUDGET Reach Project, IEEE, Engineering Mentorship Council
Programming Languages: Python, C/C++, HTML, R, Verilog, Assembly, Ladder Logic
Engineering Tools: Git, GDB, Rstudio, Visual Studio, PyCharm, Ghidra, Wireshark, VMware, Linux

Sutton Elliott is a senior Computer Engineering student at Texas A&M University with a minor in Mathematics. His academic career has seen him develop strong foundational skills in both hardware and software systems. His coursework has spanned throughout machine learning, embedded systems, databases, and software development. Sutton has also earned a Microsoft Azure AI certification, demonstrating an interest in cloud-based artificial intelligence technologies. His technical interests center around bridging the gap between hardware systems and intelligent software models. He is proficient in Python, C/C++, Azure environments, and SQL.

Sutton Elliott

10903 Thomasville Ln.
Frisco, TX 75033
9728213843 | sutton.elliott777@gmail.com | linkedin.com/in/suelliott77/
https://suelliott77.github.io/ | github.com/suelliott77

Education

Texas A&M University — **Aug 2022 - May 2026**
BA, Computer Engineering — *College Station, TX*

Key Skills

- **Key Skills**: Leadership, Project Management, Team Collaboration, Communication, Problem-solving
- **Technical Skills**: C++, Python, Java, Azure AI, SQL

Objective

Results-driven Computer Engineering major (May 2026) with a proven record of leadership and hands-on technical projects. Seeking a full-time position or internship where I can apply my skills in software development, machine learning, and AI to deliver innovative solutions.

Experience

Camp Eagle — **May 2023 - Aug 2025**
Group Camp Counselor

- Led groups of 10–30 students/adults, creating schedules tailored to group needs and ensuring adherence to daily activities.
- Collaborated with a team of counselors to maintain a safe and engaging environment for campers, resulting in positive feedback from visiting churches.
- Developed leadership, communication, and crisis management skills in a high-paced outdoor setting.
- Fulfilled a contract during the summers of 2023-2025, successfully managing group activities and contributing to the camp's operational goals

Projects

Microsoft Certified: Azure AI Fundamentals | AI-900

- Certification covering machine learning, computer vision, and Azure AI services, gaining hands-on experience with cloud-based AI solutions.

Google Docs Add-on | Google Apps Script, JavaScript, HTML
Texas A&M University

- Developed a custom sidebar add-on for Google Docs to streamline workflows with Find & Replace, signature insertion, and document saving/emailing.
- Integrated with Google Drive and Gmail API to automate saving and progress tracking for documents.

Professional Website | HTML, CSS, JavaScript, PHP
Texas A&M University

- Designed and developed a personal portfolio website hosted on a local server to showcase projects and experience.
- Implemented interactive pages and responsive layout to improve user experience and accessibility.

Database Dashboard | SQL, Python, JDBC, MySQL (AdventureWorks Dataset)
Texas A&M University

- Developed an interactive dashboard to analyze employee and customer data from the AdventureWorks enterprise database, integrating dynamic graphs, maps, and tables for real-time insights.
- Implemented custom SQL and JDBC functions for viewing tables, generating statistics, and visualizing relationships between departments, salaries, and sales performance.
- Designed and refined the GUI for intuitive data exploration, connecting database queries to visual components for seamless interactivity.

Leadership & Involvement

First Baptist Church, Bryan, TX
Freshman Equip Leader

- Mentored a group of college freshmen through weekly gatherings.
- Modeled integrity and servant leadership, helping students navigate faith, academics, and personal challenges.

First Baptist Church, Bryan, TX
Titus 10 Cluster Leader

- Led a weekly book study on Biblical manhood through the lens of the book of Titus, encouraging open discussion and practical application.

Tyler Rottermond is a senior Computer Engineering student at Texas A&M University with a strong focus on embedded systems, low-level software, and hardware–software integration. He has completed advanced coursework in operating systems, computer architecture, microprocessor systems design, and electric circuit theory, maintaining a 3.95 GPA. His technical experience includes developing embedded software in C/C++ and Python, working in Linux-based environments, designing and analyzing analog and digital hardware systems, and interfacing with custom hardware through device drivers and system-level code. Tyler's interests center on building reliable, efficient embedded platforms that bridge physical hardware with intelligent software, particularly in applications involving sensing, control, and real-time system behavior.

Tyler Rottermond

(469)-325-0248 | tyler.rottermond@gmail.com | https://www.linkedin.com/in/tyler-rottermond-41641726b/

Objective

Motivated senior Computer Engineering student at Texas A&M University with a 3.93 GPA, seeking a Summer 2026 internship to further develop industry experience before entering graduate school. Experienced in C++, Verilog, Python, MySQL, and microprocessor systems, with proven ability to deliver solutions in both academic and industry environments

Education

Texas A&M University — College Station, TX
Bachelor of Science in Computer Engineering (Spring 2026) GPA: 3.93

Professional Experience

Computer Engineering Intern | Northrop Grumman June | 2025 - Present
- Designed and implemented a MySQL database to organize and manage project data.
- Built a Python-based interface to streamline data handling and user interaction.
- Developed a GUI application in C# to process and display thermal camera outputs.
- Generated test data using polar coordinates to validate a coworker's analysis script.

Teacher's Assistant | Engineering and Physics Departments | Fall 2023 – Spring 2025
- Assisted students in learning Python and Linux scripting, helping with the transition from high school to rigorous college coursework.
- Conducted office hours, having several frequently returning students.
- Mentored first-year engineering students, facilitating their adaptation to university-level academic expectations.

Skills

Programming Languages: Verilog, C/C#/C++, Python, Assembly, Java
Tools / Technologies: MySQL, Visual Studio, Linux, Windows, Git, SVN
Core Strengths: Quick Learner, Critical Thinking, Problem Solving, Team Collaboration

Leadership

Founding Father & Treasurer | Triangle Fraternity Fall 2024 – Present
- Helped recruit and onboard 20+ new members, growing the chapter's presence on campus.
- Managed several thousand dollars in chapter funds, ensuring proper budgeting, nonprofit compliance, and accurate financial documentation.
- Assisted in planning and coordinating fraternity events, balancing organizational goals with financial responsibility.

Projects

-Receipt Checker: Wrote a script in Python that uses the selenium package to take in a user's email and survey code, and automatically filled out a survey to get the user a free entrée.

Sriharsha Jandhyala is a Computer Engineering student at Texas A&M University with experience in software development, data analytics, and applied machine learning. He has built data pipelines and full-stack applications through internships and technical projects, including financial dashboards at CBRE and a restaurant point-of-sale system. He has worked on multiple projects involved in machine learning, including fine-tuning transformer models for text classification using modern optimization techniques. His technical skill set includes Python, C++, SQL, JavaScript frameworks, SDLC tools such as Azure DevOps, and cloud and data tools such as Snowflake and Tableau. Sriharsha is passionate about building data and AI driven software that improves decision-making, efficiency, and overall user experience.

Sriharsha Jandhyala

Irving, TX • sriharsha.jandhyala03@gmail.com • 972-415-1050 • linkedin.com/in/sriharsha-jandhyala169 • U.S. Citizen

Education

Texas A&M University **May 2026**
College Station, TX
Bachelor of Science, *Computer Engineering*
Relevant Coursework: Software Engineering, Data Structures & Algorithms, Machine Learning, Introduction to Computer Systems, Computer Architecture, Program Design and Concepts, Linear Algebra, Discrete Mathematics
Organizations: Engineering Mentorship Council

Skills

Technical: Python, C++, C, SQL, Java, React.js, Node.js, Angular, HTML, CSS, Assembly, Verilog
Tools: AWS, Azure DevOps, Jira, Snowflake, Power BI, Google Cloud Platform, Git, GitHub, Linux, PyTorch, TensorFlow, Transformers, NumPy, Pandas, Tableau, Jupyter Notebook, Microsoft Office
Languages: English (fluent), Telugu

Experience

Software Engineering Intern – CBRE, Inc. **June 2025 – August 2025**
Richardson, TX

- Designed and deployed a Power BI dashboard processing 30,000+ Snowflake audit records, streamlining reconciliation and indirect accounting for the FinOps team
- Engineered an agentic pipeline to automate tariff data processing, eliminating 50+ hours of manual lookups, enabling advanced historical cost analysis, and reducing budget overruns by over 25%
- Built a React.js web interface for investment management insights, delivering portfolio analytics and a leasing office exposure map to assess geographic risk across 200+ properties in real time

Global Career Accelerator – Coding for Data **May 2024 – July 2024**

- Wrangled over 16 real-world datasets using SQL, Python, and Tableau, generating actionable insights into demographic trends, energy consumption, and market strategies
- Analyzed over 2,700 website traffic records from Grammy.com and RecordingAcademy.com, comparing user behavior before and after the domain split to identify engagement improvements
- Evaluated energy production and consumption metrics across 50+ U.S. regions to determine an optimal location for a new Intel data center

Projects

Boba Tea Restaurant Point-of-Sale System **February 2025 – May 2025**

- Developed a full-stack POS system for a boba tea restaurant with employee and manager views, enabling order processing, menu management, and drink customization with 15+ options per item
- Built a React and CSS frontend with Express.js APIs and PostgreSQL, adding high-contrast accessibility and multi-language support across 50+ languages
- Delivered a more accessible and inclusive POS experience for visually impaired and multilingual customers through iterative UI improvements in Agile sprints tracked in Jira

Fine Tuning Mistral 7B for News Article Classification **August 2024**

- Processed news article data from over 2,000 sources and employed advanced tokenization techniques
- Implemented QLoRA with 4-bit quantization to fine-tune Mistral 7B and improved scalability by reducing the number of trainable parameters
- Integrated QLoRA into the Transformers library to optimize training speed, effectively classifying articles into various genres

William Xu is a senior Computer Engineering student at Texas A&M University with expertise spanning distributed systems, robotics, and full-stack development. He has completed rigorous coursework in computer architecture, data structures and algorithms, digital system design, and artificial intelligence while gaining hands-on experience across industry and research settings. His technical background includes developing embedded software and hardware simulations in C++, Python, and SystemVerilog, building scalable mobile applications with React Native and cloud infrastructure, and implementing real-time robotics algorithms in ROS2. William's professional experience ranges from architecting distributed neural network systems and high-performance computing pipelines to developing autonomous underwater vehicle software and deploying production mobile applications. His interests lie in building intelligent, scalable systems that integrate hardware and software specifically in robotics, real-time processing, and distributed architectures where performance and reliability are critical.

William Xu

(530)-760-6746 | williamsxuu@gmail.com | www.linkedin.com/in/william-xuuu | github | portfolio

Education

Texas A&M University — College Station, TX
Computer Engineering B.S — May 2026
Relevant Coursework: *Computer Architecture, Computer Systems, Data Structures & Algorithms, Digital System Design, Software Engineering, Microprocessor Systems Design, OOP, Databases, Artificial Intelligence*

Experience

Co-Founder | *React Native, TypeScript, Firestore, AWS, Expo, Git* — September 2025 – January 2026
Pier — *College Station, TX*

- Built and sold a friend-powered dating platform enabling users to matchmake for friends, developing cross-platform mobile application using React Native and Expo for iOS and Android deployment
- Architecting scalable backend infrastructure with Firebase Firestore and AWS services, implementing real-time data synchronization and user authentication systems to support 5000+ users

Undergraduate Researcher | *Python, C++, ROS2, Docker, Git* — December 2023 – January 2026
Texas A&M University — *College Station, TX*

- Developed SLAM algorithm in ROS2 utilizing Gaussian Process regression for sonar and IMU sensor fusion, improving underwater robot localization accuracy by 30% in low-visibility environments
- Built modular sensor fusion pipeline with Kalman filtering for processing noisy sonar data, enabling real-time localization at 20Hz update rates for underwater robotics applications

R&D Intern | *SystemVerilog, Python, Verilator, Git, Assembly* — June 2025 – September 2025
Nano IC — *Milpitas, CA*

- Developed Python simulation framework for distributed neural network architecture, implementing packet routing algorithms and BF16 arithmetic operations to validate hardware designs before FPGA synthesis
- Created C++ behavioral models for hardware components including CORDIC arithmetic units and floating-point processors, enabling 100x faster design validation compared to RTL simulation

R&D Contractor | *Go, Python, Kubernetes, gRPC, Git* — May 2025 – August 2025
Roblox — *San Mateo, CA*

- Engineered dynamic server rebalancing system for distributed game infrastructure, implementing intelligent workload migration algorithms that reduced server hotspots by 20% and improved resource utilization across data centers
- Optimized high-performance computing pipelines using gRPC and Kubernetes orchestration, achieving 35% reduction in P99 latency for real-time game session processing through efficient service mesh architecture and request batching

REU Research Intern | *Python, C++, ROS2, Docker, Git, Foxglove* — May 2024 – August 2024
NSF & Texas A&M University — *College Station, TX*

- Led field testing of BlueROV2 autonomous underwater vehicle across 15+ scenarios, improving navigation success rate by 25% in confined spaces through enhanced sonar perception algorithms
- Implemented GitLab CI/CD pipelines with automated testing frameworks for robotic software deployment, accelerating development cycles by 25% while ensuring code quality

Technical Skills

Programming Languages: Python, C++, Go, JavaScript, TypeScript, SystemVerilog, SQL, C, Assembly
Frameworks & Libraries: React Native, ROS2, gRPC, Expo, NumPy, pandas, OpenCV, TensorFlow, PyTorch
Tools & Platforms: Git, Docker, Kubernetes, AWS, Firebase/Firestore, PostgreSQL, WSL, GitLab CI/CD, Foxglove
Methodologies: Distributed Systems, High-Performance Computing, CI/CD, DevOps, Mobile Development, Agile, Test-Driven Development